

ReLoop

24-Hour Build Plan · v2 · iOS + Ray-Ban Meta + Dimensional robot dog

One-liner: Send the robot dog to scout your place while you're out (or walk it once yourself in Ray-Ban Meta glasses). ReLoop figures out what's worth selling, donating or recycling, and once you tap Approve, a seller agent keeps negotiating after everyone has walked away.

At a glance	
Team	4 people — lanes A, B, C, D. A, B, C have a Mac + iPhone. D does not.
Clock	24 hours. Core by 3:30, walk-away proof by 5:00, everything after is additive.
Hardware	Ray-Ban Meta glasses (no display) via Meta Wearables Device Access Toolkit (DAT) for iOS; Dimensional robot dog (Unitree Go2 running DimOS — confirm model at hour 0).
Client	Native iOS app (Swift / SwiftUI). No Kotlin, no Android.
Server + robot	Python everywhere off the phone: FastAPI server, and the DimOS robot module (DimOS is Python-native). One language for B, C and D.
Demo table	8 real items: PS4 Slim, wireless headset, 24" monitor, old phone, keyboard, desk lamp, bin of cables, dead phone. Put some on the floor so the dog can see them.

What changed from v1

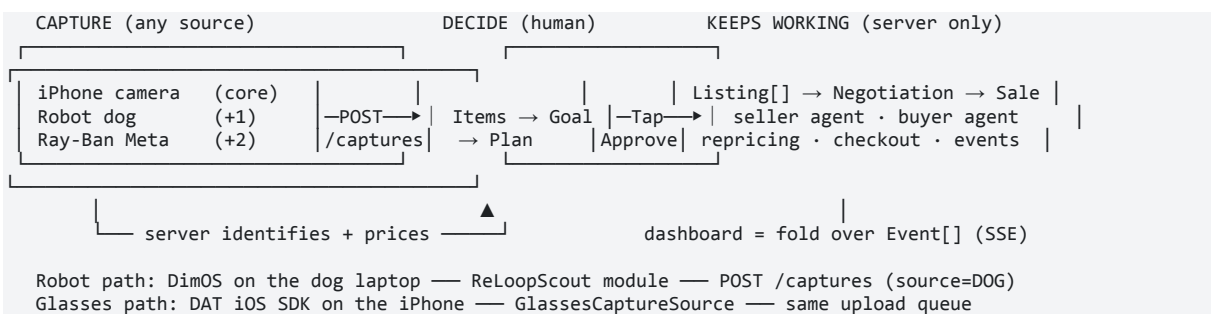
Change	Consequence in this plan
Glasses have no display	No HUD, no Neural Band taps. Glasses = camera + open-ear speakers. All decisions happen on the phone; status is spoken through the glasses. DAT work shrinks from ~5 hrs to ~3.
Robot dog from Dimensional	New capture source that works while you're away. It is Addition #1 — the first thing built after the core.
iOS instead of Kotlin/Android	SwiftUI client, DAT iOS SDK (Swift Package). Server and buyer agent move to Python so D (no Mac) can contribute to everything except the app.
24 hours, not 36	Core compressed to 3.5 hrs, walk-away proof by 5:00, freeze at 21:00. Tracks are cheap modules on top, not new architecture.
New track list	Sustainability first, then Visa, Ramp, Meta, Dimensional, Long Lake. Voloridge dataset is an optional extra at the very end.

1. The shape of the build

Three rules make a 24-hour plan with two pieces of preview hardware survivable:

1. **The core never needs hardware.** Phone camera in, store and seller agent out. Built and demoable by 3:30 with nothing but iPhones and a server.
2. **Every piece of hardware is just another capture source.** The dog and the glasses both produce the same thing the phone does — a Capture (a photo plus where it came from). The server, planner, store and agents never learn which device took the picture.
3. **Selling lives on the server.** The phone can close, the glasses can go back in the case, the dog can sit down, and negotiation keeps running. That is the product.

Architecture



The dog does **not** talk to the phone. It talks to the server, exactly like the phone does. The phone learns about dog finds the same way it learns about sales: an event over SSE. That keeps the robot lane (D) independent of the iOS lane (A).

Repo layout

```
reloop/  
  contracts/  schema.py (pydantic, source of truth) + fixtures/*.json  
  server/    FastAPI: identify, plan, store, catalog, seller agent, workers, dog missions, SSE  
  buyer/    Python CLI buyer agent (scripted + LLM modes)  
  web/       dashboard + storefront pages (server-rendered HTML + a little JS) ← D  
  robot/     DimOS ReLoopScout module + scout.py patrol runner ← D  
  ios/       ReLoop.xcodeproj – SwiftUI app, Codable mirror of contracts ← A  
  data/      price_book.csv, routing_rules.yaml, dropoff_sites.json
```

2. Scope and honesty rules

Judges probe claims. The precise version of each claim is the more impressive one.

- **The dog proposes, the human approves.** The dog finds and photographs candidates; nothing is listed until a person taps Approve on the phone. Say this on stage — it is also your Long Lake answer.
- **The dog scouts a prepared area.** It patrols a mapped demo zone with a few tagged spots. It is not free-roaming a house. Autonomous exploration is a stretch goal, labelled as such.
- **The store is real; the demo buyer is ours.** Every approved item gets a public page, `/catalog.json`, and `schema.org` Product JSON-LD. Our buyer agent uses the same public API an outside agent would.
- **The seller agent can't give your stuff away.** Floors live server-side, are never serialized, and are enforced in code after the model speaks.
- **Prices are estimates** from a hand-researched price book with source and date on every row. Every listing shows its basis.
- **Glasses capture only on an explicit tap** on the phone. No continuous capture of other people in a shared venue.

Permanently out of scope: posting to real marketplaces, a carbon model, perfect vision, a giant dashboard, the dog picking things up.

3. Lanes

Lanes follow the hardware. A owns the iPhone app and the glasses. D, the one person without a Mac, owns everything that runs in a browser or on the robot — both work fine without Xcode.

Lane	Machine	Owens	Hard deliverable
A · App	Mac + iPhone	ios/ — SwiftUI client, upload queue, Scout tab, DAT glasses	Scan → Review → Goal → Plan → Approve on the phone by 3:30. Dog inbox by 8:00. Glasses capture by 12:00.
B · Brain	Mac + iPhone	/identify, pricing, /plan, price_book.csv, all non-negotiation prompts	Photo → priced Item[] → Plan. Works on phone, dog and glasses frames (dog frames are the hard ones).
C · Market	Mac + iPhone	server core, events/SSE, store API, catalog, seller + buyer agents, checkout, dog-mission API	Scripted sale by 3:30; LLM buyer + walk-away test by 5:00; /dog/missions by 7:00.
D · Robot + Web	Any laptop (Ubuntu ideal for DimOS)	robot/ (DimOS), web/ (dashboard + storefront), demo script & video	Dashboard live by 3:30. Dog patrols 3 spots and posts captures by 8:00. Pitch video by 20:00.

Bottlenecks. A is the thinnest lane — one iOS developer, two SDK-dependent features. B and C both have Macs and iPhones and should be able to build and run the app, so whoever finishes first pairs with A on the glasses. D's risk is robot bring-up on venue Wi-Fi, which is why D starts it at minute zero in the background while building the dashboard.

D's laptop. DimOS recommends Ubuntu 22.04/24.04 with an NVIDIA GPU; macOS is experimental. If D has Windows, decide before the event: Ubuntu on a USB/dual-boot, or ask Dimensional whether they supply a robot laptop. WSL2 is not a safe bet for WebRTC/BLE to the robot. If none of that works, D drives DimOS from B's Mac (experimental) and keeps owning the code.

Lane A · App (iOS)

- **Owens:** ios/. SwiftUI, AVFoundation camera, offline upload queue, SSE client, text-to-speech, DAT integration.
- **Inputs:** nothing — start of the chain. **Outputs:** Captures, answers to questions, a Goal, the Approve tap, a “Send the dog” request.
- **Checkpoints:** 3:30 full loop on the phone · 5:00 survives force-quit and Wi-Fi drop · 8:00 Scout tab shows dog finds · 12:00 glasses photo in the same queue.
- **Never:** put selling logic in the app; block the UI on the glasses; require the glasses for any core screen.

Lane B · Brain (perception, pricing, planning)

- **Owens:** /identify, pricing, /plan, price_book.csv, routing_rules.yaml, every prompt except negotiation.
- **Checkpoints:** 3:30 real photos → priced items · 5:00 questions + all five goal modes · 9:00 dog frames identified with dedupe across frames · 14:00 Sustainability + Ramp numbers.

- **Never:** let the model pick prices or actions; accept a category not in the book; scrape anything at runtime.

Lane C · Market (server, agents, commerce)

- **Owens:** FastAPI app, SQLite event log, SSE, approve → listings, catalog, seller agent + floor policy, buyer CLI, workers, checkout adapters, /dog/missions.
- **Checkpoints:** 0:45 hello-world deployed at the real URL · 3:30 scripted sale · 5:00 walk-away test passes · 7:00 dog mission API live · 15:00 Visa checkout or honest mock.
- **Never:** return SellerRules from any route; allow unbounded negotiation rounds; run a worker anywhere but the server.

Lane D · Robot + Web

- **Owens:** robot bring-up, robot/ (DimOS module + patrol runner), web/ (dashboard, storefront pages), demo script, backup video.
- **Checkpoints:** 1:00 dog on the network, replay/sim runs · 3:30 dashboard from events · 5:00 dog maps the demo zone with 3 tagged spots · 8:00 first dog mission end to end · 11:00 dog gate.
- **Never:** put pricing or selling logic in the robot code; let the dog move without a spotter holding the stop control.

4. Data contracts (freeze at 0:45)

contracts/schema.py is the source of truth. A mirrors it as Swift Codable structs, and a unit test in Xcode decodes every file in contracts/fixtures/ — if a field changes, the iOS test goes red immediately instead of at hour 16. FastAPI's OpenAPI output is the reference for anyone confused.

```
# contracts/schema.py
class Source(str, Enum): PHONE="PHONE"; GLASSES="GLASSES"; DOG="DOG"; FIXTURE="FIXTURE"

class Capture(BaseModel):
    id: str # "cap_001"
    ts: datetime
    source: Source
    uri: str # "blob://cap_001.jpg"
    spot: str | None = None # DOG: tagged location, e.g. "desk", "shelf"
    missionId: str | None = None # DOG: which scout mission took it

class Question(BaseModel): text: str; options: list[str] # "Is this a PS4 Slim?"

class Item(BaseModel):
    id: str; captureIds: list[str]; label: str
    category: str # price_book key, e.g. "console.ps4_slim"
    condition: Condition # LIKE_NEW | GOOD | FAIR | POOR | BROKEN
    confidence: float
    foundBy: Source # shown in UI: "Found by the dog near the desk"
    question: Question | None = None
    confirmedByUser: bool = False

class PriceBand(BaseModel): quickUsd: int; normalUsd: int; maxUsd: int; basis: str
class Goal(BaseModel): mode: Mode; targetUsd: int|None=None; deadline: datetime|None=None
    upgradeTo: str|None=None; budgetUsd: int|None=None
class Decision(BaseModel): itemId: str; action: Action; band: PriceBand|None=None
    listUsd: int|None=None; reason: str
class Plan(BaseModel): id: str; goal: Goal; decisions: list[Decision]
    expectedUsd: int; goalGapUsd: int; approved: bool=False
class PublicListing(BaseModel): id: str; itemId: str; title: str; description: str
    photoUrl: str; listUsd: int; status: ListingStatus

@dataclass # NOT a pydantic model on purpose: no route can return it
class SellerRules: listingId: str; floorUsd: int; autoAcceptUsd: int
    maxRounds: int = 4; urgency: float = 0.0

class NegotiationMsg(BaseModel): threadId: str; listingId: str; frm: Party; kind: MsgKind
    priceUsd: int|None=None; text: str; ts: datetime

class DogMission(BaseModel):
    id: str; status: MissionStatus # QUEUED | RUNNING | DONE | FAILED
    spots: list[str] # ["desk", "shelf", "floor_corner"]
    captureIds: list[str] = []

class Event(BaseModel): # append-only; the dashboard is a fold over these
    ts: datetime; kind: EventKind # IDENTIFIED PLAN_APPROVED LISTED OFFER COUNTER SOLD
    # DONATED RECYCLED REPRICED DOG_STARTED DOG_FOUND DOG_DONE
    listingId: str|None=None; amountUsd: int|None=None; text: str

// ios/ReLoop/Models.swift (mirror – decoded against contracts/fixtures in ContractsTests.swift)
enum Source: String, Codable { case PHONE, GLASSES, DOG, FIXTURE }
struct Capture: Codable, Identifiable { let id: String; let ts: Date; let source: Source
    let uri: String; var spot: String?; var missionId: String? }
```

Two decisions worth defending out loud: SellerRules is not serializable, so the floor cannot leak through a careless encoder; and the dashboard stores nothing — recovered dollars, goal gap, items reused and “agent still working on N” are all folds over Event[], so replaying the log rebuilds the demo.

5. The engine (carried over from v1, trimmed)

The model identifies; code decides. The LLM answers “what is this, what condition, how sure are you.” Pricing, routing and the plan are deterministic functions over the price book — testable and explainable.

Identification (B)

- Claude vision with a structured output schema: a list of {label, category, condition, confidence}; category must be a price-book key or other.unknown.
- Confidence < 0.75 or variant ambiguity (Slim vs Pro) → a Question with 2–3 options, shown on the phone.
- Dedupe within a scan: same category + near-identical label → merge and keep both captureIds. This matters far more for the dog, which will see the same monitor from three angles.
- **Dog-specific:** dog frames are low, wide and cluttered. Ask the model to ignore people, fixed furniture and anything not plausibly resellable, and to return nothing rather than guess. Prefer one good frame per spot over many bad ones.

Pricing and routing (B)

```
category,label,normal_usd,spread_pct,min_listable_usd,e_waste,source,researched
console.ps4_slim,Sony PS4 Slim,185,13,20,true,"recent sold listings n=12",2026-09-xx
```

```
condMult = {LIKE_NEW:1.10, GOOD:1.00, FAIR:0.80, POOR:0.55, BROKEN:0.15}
normal = round5(book.normal * condMult)          # 185
quick  = round5(normal * (1 - spread))            # 160
max    = round5(normal * (1 + spread))            # 210
```

Rule (first match wins)	Action
User marked “keep”	KEEP — always wins
BROKEN and e_waste	RECYCLE — nearest e-waste drop-off
normal < min_listable (not YARD_SALE)	DONATE — not worth a negotiation
Mode ZERO_WASTE	Nothing defaults to KEEP
Everything else	SELL

The planner is greedy and deterministic (no LLM): pick the price tier from goal and deadline, set listUsd a few percent above tier for negotiating room, set floorUsd from the quick price and urgency. Every Decision carries a human-readable reason. Upgrade Mode (Ramp) is the same planner with a derived target: price of the thing you want minus your out-of-pocket budget.

Negotiation (C)

```
def respond(m, listing, rules, round):
    p = m.priceUsd
    if p is None: return seller_llm.inform(m, listing)
    if p >= listing.listUsd or p >= rules.autoAcceptUsd: return accept(p, "Deal.")
    if round >= rules.maxRounds:
        return accept(p, "Deal.") if p >= rules.floorUsd else reject("Can't go that low, sorry.")
    s = seller_llm.propose(m, listing, round) # {move, priceUsd, text}
    if s.move == "ACCEPT" and p >= rules.floorUsd: return accept(p, s.text)
    c = clamp(s.priceUsd, max(p + 1, rules.floorUsd), listing.listUsd) # the clamp
```

```
return counter(c, s.text)
```

Max four rounds, a per-thread timeout, one hold per listing, everything is an event, and a truth-table test of the policy that runs without an LLM. Workers: **SellerWorker** (event-driven) and **RepricingJob** (ticks, raises urgency on stale listings). `RELOOP_TIME_SCALE=60` makes a minute an hour for the demo, shown on the dashboard.

Canonical transcript in `fixtures/`: buyer asks for a PS4 under \$185 → seller counters \$180 from a \$190 list → buyer offers \$175 → seller accepts (floor \$165) → SOLD \$175 → dashboard: \$125 left to goal.

6. The timeline

Block	Clock	What exists at the end
Phase 0 · Contracts	0:00–0:45	Schemas frozen, fixtures written, server deployed, Xcode project decoding fixtures. Dog powered and joining the network in the background.
Phase 1 · Core loop	0:45–3:30	◆ core-v1: phone photo → items → goal → plan → Approve → listing → scripted buyer → SOLD → dashboard.
Phase 2 · Make it real	3:30–5:00	◆ MVS: LLM buyer, server workers, walk-away test passes. A minimum viable submission. Everything after is additive.
Addition 1 · Robot dog	5:00–11:00	“Send the dog” from the phone → dog patrols tagged spots → captures → items in the phone’s inbox → Approve. Gate at 11:00.
Addition 2 · Glasses	7:00–13:00	Ray-Ban Meta photo into the same queue; spoken status through the glasses. Gate at 13:00.
Additions 3+ · Tracks	11:00–18:00	Track modules in priority order (section 9). Sleep shifts start.
Real run + record	18:00–21:00	Full run with dog + glasses + real table. Event log saved as seed. Backup video.
Freeze + rehearse	21:00–24:00	Crash fixes only. Rehearse ×3. Submit early.

Phase 0 — Contracts (0:00 → 0:45)

- Repo up; server/ runs locally and is deployed to the real URL (hello-world). Do this now, not at hour 20.
- B + C write `schema.py`; A writes the Swift mirror + `ContractsTests` decoding fixtures.
- Fixtures: 8 photos of the demo items, `items.json`, `plan.json` (“\$300 by tomorrow”), full negotiation transcript, and **one dog mission** with 3 dog-angle photos (take them with a phone held at 40 cm off the floor).
- D: power the dog, `dimos go2tool discover / connect-wifi`, confirm ping < 10 ms, start `dimos --replay run unitree-go2` in the background, then switch to the dashboard.

Exit test: `python -m server.demo_loop` plays fixtures end to end and prints `SOLD $175 · $125 left`
 • agent working on 5.

Phase 1 — Core loop on the phone (0:45 → 3:30)

- **A:** `PhoneCaptureSource` (AVFoundation), upload queue, five screens: Scan (shutter + running total) → Review (questions as buttons) → Goal (preset chips) → Plan (totals + reasons + Approve) → Live (native summary + WebView of the dashboard).
- **B:** `POST /identify` (multi-item, constrained categories) and pricing; `POST /plan` in CASH mode.
- **C:** approve → listings + rules + LISTED events; `GET /catalog.json`; `POST /threads/{id}/messages` with the policy; scripted buyer CLI; `MockCheckout` → SOLD; SSE stream of events.
- **D:** storefront page per listing and the dashboard, both rendered from the event log: goal bar, recovered \$, sold cards, transcript, “agent still working on N”.

At 3:00 everyone photographs the real demo items with the barely-working app so B tunes on real photos, not stock images, for the next 20 hours.

◆ **3:30 core-v1**: “I photographed eight things, approved a plan, a buyer bot bought the PS4, the dashboard says \$125 to go.” Tag it.

Phase 2 — Make “keeps working” a fact (3:30 → 5:00)

- **C**: LLM buyer with four tools (`search_catalog`, `get_listing`, `send_message`, `checkout`); SellerWorker + RepricingJob as server background tasks; PENDING holds; policy table tests.
- **B**: clarifying questions end to end; dedupe; condition grading; listing copy; all five goal modes in routing and tiering.
- **A**: offline queue survives Wi-Fi drop and force-quit; Question screen driven by `Item.question`.
- **D**: dashboard readable from across a room; JSON-LD on store pages; then **map the demo zone with the real dog** and tag three spots (see section 7).

◆ **5:00 walk-away test / MVS**: approve a plan, force-quit the app, start the LLM buyer from another laptop, watch SOLD land on the dashboard. If this passes, you have a submission. Everything after this line is upside.

7. Addition 1 — the robot dog scout

The story: you leave for the day and tap “Scout while I’m out.” The dog walks your space, stops at each spot, looks, and photographs anything that could have a second life. When you check your phone: “The dog found 5 things worth about \$412. Review?” You approve from the couch — or from work — and the seller agent takes it from there.

What Dimensional gives us (from the DimOS docs)

- **DimOS** is a Python SDK / “agentic OS” for robots; no ROS required. The Unitree Go2 (Pro/Air) is its most stable platform and connects over WebRTC on stock firmware 1.1.7+ — no jailbreak.
- **Blueprints** are prebuilt stacks you run with the dimos CLI: unitree-go2 (full navigation: SLAM, costmap, A* planning, frontier exploration, command center at localhost:7779), unitree-go2-agentic (navigation + LLM agent + MCP), unitree-go2-spatial, unitree-go2-detection.
- **Skills** are Python methods decorated with `@skill` on any Module. The agent discovers them automatically and they are exposed as MCP tools (default port 9990). Built-ins we use: `navigate_with_text(query)`, `tag_location(name)`, `move_to(...)`, `observe()`, `speak(text)`, `stop_navigation()`.
- **CLI control:** `dimos run ... --daemon`, `dimos status`, `dimos log -f`, `dimos agent-send "..."`, `dimos mcp list-tools`, `dimos mcp call <skill> --arg k=v`, `dimos stop`.
- **No hardware needed to start:** `dimos --replay run unitree-go2` (recorded session) and `dimos --simulation run unitree-go2-agentic` (MuJoCo + MCP).
- **Agent LLM:** the default agent model uses `OPENAI_API_KEY`; any LangChain-compatible model can be set on `McpClient.blueprint(model=...)`. Our deterministic patrol path needs no agent LLM at all.

Design: the dog is a capture source that dials out

We add one small DimOS module and one runner. Nothing in DimOS is modified.

```
# robot/reloop_scout.py - a DimOS Module that adds one skill
class ReLoopScout(Module):
    color_image: In[Image] # autoconnected to the Go2 camera stream

    @rpc
    def start(self):
        self._latest = None
        self.color_image.subscribe(lambda img: setattr(self, "_latest", img))

    @skill
    def snapshot_for_resale(self, spot: str, note: str = "") -> str:
        """Photograph what is in front of the robot so ReLoop can check whether anything
        here could be sold, donated or recycled. Call once per spot, after stopping."""
        cap = post_capture(self._latest, source="DOG", spot=spot,
                           mission_id=CURRENT_MISSION) # POST /captures
        return f"sent {cap.id} from {spot}"

# blueprint: the stock Go2 agentic stack + our module
reloop_go2 = autoconnect(unitree_go2_agentic, ReLoopScout.blueprint())

# robot/scout.py - deterministic patrol runner (the reliable path)
while True:
    m = GET /dog/missions/next # the dog polls the server; no inbound ports
    if not m: sleep(2); continue
    POST /dog/missions/{m.id}/start # -> DOG_STARTED event -> phone + dashboard
```

```

for spot in m.spots:
    call_skill("navigate_with_text", query=spot)      # ["desk", "shelf", "floor_corner"]
    call_skill("snapshot_for_resale", spot=spot)      # via DimOS MCP (port 9990)
    call_skill("speak", text=f"Checking the {spot}") # optional, fun on stage
call_skill("navigate_with_text", query="home")
POST /dog/missions/{m.id}/done                      # → server identifies → DOG_FOUND events

```

`call_skill` talks to the DimOS MCP server. If the MCP client fights you, shell out to `dimos mcp call <skill> --arg ...` — ugly, fine for a hackathon. The dog polls the server rather than the server calling the dog, so venue NAT and firewalls don't matter.

Server and app side

- **C:** POST `/dog/missions` (from the phone), GET `/dog/missions/next, .../start, .../done`, and POST `/captures` accepting `source=DOG`. On done, run identification over the mission's captures and emit `DOG_FOUND` events.
- **B:** identification tuned for dog frames: low angle, clutter, repeated views. Dedupe across the whole mission. Label each item with its spot ("Found by the dog near the desk").
- **A:** a Scout tab — "Scout while I'm out" button, spot checklist, live mission status from SSE, and a Found inbox that drops straight into the existing Review → Goal → Plan flow.
- **D:** dashboard gets a dog panel: mission status, the dog's frames per spot, and "Found by the dog" badges on listings.

Build order — don't move on until each step works

1. **Bring-up (D, hour 0, background).** Dog on the same network as the dog laptop; ping < 10 ms, 0% loss; onboard obstacle avoidance on; replay runs.
2. **Drive + see (D, by 5:00).** `dimos run unitree-go2` on the real dog; watch the camera in the command center; click-to-navigate across the demo zone.
3. **Map + tag (D, 5:00–5:45).** Drive to desk, shelf, floor corner and home; tag_location each. Confirm `navigate_with_text("desk")` returns there.
4. **One photo (D + C, by 6:30).** `ReLoopScout.snapshot_for_resale` posts a real dog frame to `/captures`. The server sees `source=DOG`. This single step is the whole Dimensional story; if you only get here, you still have it.
5. **One mission (all, by 8:00).** Phone → `/dog/missions` → `scout.py` → 3 spots → captures → items in the phone's Found inbox → Approve → listed.
6. **Voice + polish (D, 8:00–11:00).** `speak()` at each spot and "Found a PS4 — adding it" after identification; robustness (retry a failed nav, skip a spot, always return home).
7. **Stretch: agentic scout.** `dimos agent-send "explore this room and call snapshot_for_resale at anything that looks resellable"`. Show it only if it behaves; the deterministic patrol stays the demo path.

Dog gate at 11:00. If the real dog cannot complete a mission, fall back in this order: (1) teleop the dog through the spots by hand while `snapshot_for_resale` still posts real frames; (2) run the same mission in `--simulation` on the projector plus real dog-angle fixture photos. The rest of the product is untouched either way.

Dog safety and etiquette

- One named spotter holds the robot's stop control whenever it moves. Nobody else commands it.
- Patrol only inside a taped-off demo zone; keep onboard obstacle avoidance on; slow planner speed.
- Dog frames that contain people are dropped server-side before identification, and never shown on the public store.
- Charge the dog between runs; know its battery life before hour 18, not during judging.

8. Addition 2 — Ray-Ban Meta glasses (no display)

Without a display the glasses become what they're good at: a camera at eye level for the things a dog can't see (shelves, closets, desk drawers) and open-ear speakers for status. **The dog covers the floor; the glasses cover everything above it.**

What the DAT iOS docs say we need

- Swift Package <https://github.com/facebook/meta-wearables-dat-ios>; import MWDATCore and MWDATCamera. Supported glasses include Ray-Ban Meta (Gen 1 and Gen 2). iOS 15.2+.
- Info.plist: MWDAT dict with AppLinkURLScheme, MetaAppID, ClientToken, TeamID; a matching CFBundleURLTypes Scheme; UIBackgroundModes → bluetooth-central; NSBluetoothAlwaysUsageDescription; NSLocalNetworkUsageDescription + NSBonjourServices (Wi-Fi streaming); NSCameraUsageDescription.
- The app must use a **registered bundle identifier** (Apple Developer account). The SDK uses ExternalAccessory, so it can't ship to the App Store yet — we install from Xcode, which is fine.
- Flow: Wearables.configure() at launch → startRegistration() (bounces through the Meta AI app, back via handleUrl in .onOpenURL) → checkPermissionStatus(.camera) / requestPermission(.camera) → createSession(deviceSelector: AutoDeviceSelector(...)) → session.start() → addCamera(config:) → stream.start() → photoDataPublisher.listen + capturePhoto(format: .jpeg).
- Stream config: resolution .low (360×640), .medium (504×896) or .high (720×1280); frame rate 2, 7, 15, 24 or 30. Bandwidth is limited and frames are compressed adaptively — request low or medium and a low frame rate for cleaner stills.
- **Mock Device Kit** for iOS lets A build all of this without glasses; Developer Mode in the Meta AI app lets an unpublished app register without review.

Design

```
protocol CaptureSource { var isAvailable: Bool { get }
                          func capturePhoto() async throws -> Data }
final class PhoneCaptureSource: CaptureSource { /* AVFoundation - core */ }
final class GlassesCaptureSource: CaptureSource { /* DAT stream + capturePhoto - addition 2 */ }

protocol StatusSpeaker { func say(_ text: String) } // AVSpeechSynthesizer; plays through
                                                    // the glasses when they're the audio route
```

- The Scan screen gets a source toggle: **Phone / Glasses**. The shutter button stays on the phone (no display, no Neural Band). Photos go into the same upload queue; the server can't tell the difference except for source.
- **Spoken status:** after identification, “PS4 Slim, about \$185.” After a sale, even hours later: “Sold the PS4 for \$175. \$125 to go.” That makes the keeps-working moment audible to the room.
- Voice goal via the glasses mic is a late optional module, not the input path. A loud venue plus a low-bandwidth mic makes tapping the phone the reliable path.

Build order

1. Mock Device Kit wired in; GlassesCaptureSource compiles and returns a mock frame (A, from 7:00).

2. Real registration round-trip through the Meta AI app; camera permission granted.
3. Session reaches started; camera stream reaches streaming at low/7 fps.
4. One `capturePhoto(format: .jpeg) → bytes → existing upload queue`. **This step is the whole glasses story.**
5. Handle disconnects (Bluetooth drops are normal); speak status through the glasses.

Glasses gate at 13:00. If DAT still fights you, ship the phone camera. The dog is already the “walk away” story; the glasses are a nice-to-have on top.

9. Tracks → additive modules

Each module ships behind a flag, depends only on the core, and can be cut without breaking anything. Pull in this order from 11:00; one module per person at a time; merge only if the walk-away test still passes.

#	Track	Honest framing	What ships	Owner	~Hrs
T1	Sustainability	Usable stuff is trashed because selling is work. We removed the work, so reuse beats the bin.	Impact screen (scanned · value found · listed · donated · recycled · routed to reuse), drop-off list (static JSON for the area), ZERO_WASTE mode as the default demo goal.	B + D	2
T2	Visa · Reimagine Shopping	Normal people become merchants for AI agents; buying is agent-to-agent with a human-set floor.	Catalog + LLM buyer already exist. Add real checkout behind CheckoutAdapter (Visa sandbox/APIs if the sponsor provides access at hour 0, else Stripe test mode, else honest mock). An outside agent buys via the MCP catalog.	C	3
T3	Ramp · Save time & money	Listing 8 items by hand is an evening; ReLoop is a few taps.	“Time saved” counter (stated minutes-per-listing assumption × items listed + messages the agent handled) and Upgrade Mode: “PS5 for about \$125 out of pocket.”	B (+A screen)	2
T4	Meta · People closer together	Your old things go to people near you, first to people you know.	Glasses capture (Addition 2). “Friends first” — before public listing, share a link via the iOS share sheet; friends get a 1-hour head start and a friend price. Donations routed to a local person/org.	A + C	2
T5	Dimensional	A robot that turns your space into a store while you’re gone.	Addition 1 — already built. Add the dog panel on the dashboard and a clean 30-second dog segment in the video.	D	—
T6	Long Lake · Convince a non-believer	AI that shows its work and can’t go rogue with your stuff.	“Why?” on every decision (reason + price basis), a visible floor policy (“the agent can’t go below \$165 — that’s code”), an audit log of every agent message, and the human Approve gate. Mostly pitch + one screen.	D + B	1.5
T7	Voloridge · Signal in the noise (extra)	Prices grounded in a large real dataset.	Only if everything else is done: use a large public product dataset (e.g. Amazon Reviews 2023 item metadata — verify license and size before the event) to seed or sanity-check price-book categories offline. Never at runtime.	B	2–3

Default pull order. A: glasses → Friends first → Upgrade screen. B: dog identification → T1 → T3 → T6 reasons → T7. C: dog API → T2 checkout → MCP catalog → Friends first server. D: dog → dog dashboard panel → T1 screen → T6 screen → video.

10. Lanes, hour by hour

Time	A · App (iOS)	B · Brain	C · Market	D · Robot + Web
0:00–0:45	Xcode project, SwiftUI shell, Codable mirror, ContractsTests	schema.py with C; price book (8 rows); identify prompt	FastAPI + SQLite event log; deploy hello-world; transcript fixture	Dog on network, ping, replay running; web skeleton
0:45–2:30	PhoneCaptureSource; Scan screen; upload queue	/identify (multi-item, constrained); pricing	Approve → listings + rules; catalog.json; message route + policy	Storefront page + dashboard from events
2:30–3:30	Review, Goal, Plan, Live screens. 3:00 photograph the real items	/plan (CASH); tune on real photos	Scripted buyer; MockCheckout → SOLD; SSE	Dashboard live over SSE
◆ 3:30 core-v1 — full loop on the phone				
3:30–5:00	Offline queue; force-quit test; Question screen	Questions; dedupe; condition; all 5 modes	LLM buyer; workers; holds; policy tests	Real dog: drive, see, map demo zone
◆ 5:00 MVS — walk-away test passes				
5:00–8:00	Scout tab: send dog, mission status, Found inbox	Dog-frame identification + mission dedupe	/dog/missions API; captures source=DOG; DOG_* events	Tag spots; ReLoopScout skill; scout.py patrol
◆ 8:00 First dog mission end to end				
8:00–11:00	Glasses: Mock Device Kit → registration → session	Tune on real dog frames; T1 impact numbers	T2 checkout adapter (Visa / Stripe / mock)	Dog speech, retries, return home; dog dashboard panel
◆ 11:00 Dog gate				
11:00–13:00	Glasses photo → queue; spoken status	T3 time saved + Upgrade	MCP catalog; outside agent buys	Stretch: agentic scout; T1 screen
◆ 13:00 Glasses gate				
13:00–18:00	Friends first; Upgrade screen; polish. Sleep 2 h	T6 reasons; T7 if spare. Sleep 2 h	Friends first server; soak-test agents. Sleep 2 h	T6 screen; video storyboard. Sleep 2 h
18:00–21:00	Full real run with glasses + dog	Seed data from real run; audit every price basis	Seed replay mode; soak test	Record backup video; slides; QR poster
◆ 21:00 Code freeze. Crash fixes only. Rehearse ×3, submit by 23:00				

Merge discipline: trunk-based; main must build and pass demo_loop and the iOS ContractsTests. Everyone merges at least every two hours. Each ◆ is a 10-minute standup: status, blockers, re-assign. Sleep shifts are staggered so at least two people are always awake, and the dog and glasses are never tested by someone who just woke up.

11. Risks

Risk	Mitigation
Dog can't join venue Wi-Fi (captive portal, client isolation)	Bring a travel router; dog, dog laptop and one iPhone on it; server stays on the public URL. Test at hour 0.
D's laptop can't run DimOS	Decided before the event (Ubuntu USB / Dimensional-provided machine / B's Mac as experimental fallback).
Dog navigation flaky in a crowded room	Small taped zone, three tagged spots, deterministic patrol, spotter. Fallbacks: teleop through the spots, then simulation.
Dog frames are bad for identification	Stop before capturing; one frame per spot; items placed at dog height on the demo table; B tunes on real dog frames from 8:00.
DAT on iOS eats the day	Mock Device Kit from 7:00; hard gate at 13:00; phone camera already works.
Bluetooth drops mid-demo	Observe session state, reconnect, keep captures queued. Backup video.
Apple signing / bundle ID problems	One paid Apple Developer team, bundle ID registered before the event, all three iPhones added to the team.
Misidentified items	Categories constrained to the book; questions; nothing lists without Approve.
Seller sells below floor	Floor is code, not prompt; SellerRules not serializable; clamp after every reply; table tests.
Agents slow or haggle forever	Max 4 rounds + timeout; fast model; scripted buyer fallback; seeded event log.
Batteries (dog, glasses, phones)	Charge between every run. The one failure with no software fix.

Cut list, in order

1. T7 dataset → cut
2. Agentic dog scout → deterministic patrol
3. Friends first → public listing only
4. Real checkout → mock checkout (and don't claim payments)
5. MCP catalog → /catalog.json
6. Glasses → phone camera
7. LLM buyer → scripted buyer
8. Real dog missions → teleop through spots → simulation + dog-angle fixtures

Cut all eight and you still have the 5:00 MVS: a real store, a seller agent with a hard floor, and a sale that lands after the app is closed.

12. What to say on stage (~2.5 min)

People don't throw usable things away because they're worthless. They throw them away because selling is work — pricing, photos, listings, messages, haggling. ReLoop removes the work. Even the walking around.

[Phone: tap **Scout while I'm out**. The dog stands up and walks to the desk.] "I'm leaving for the day. The dog isn't." [Dog stops at each spot: "Checking the shelf."]

[Phone buzzes: **The dog found 5 things worth about \$412.**] "It found the PS4 and the monitor. It asks me one thing — is this a PS4 Slim? Yes." [Tap.] "Goal: make \$300 by tomorrow, waste nothing." [Tap.] "Sell six, donate the cable bin, recycle the dead phone." [Tap **Approve**.]

[Put on the glasses; photograph the headset on the high shelf.] "The dog does the floor; I do the shelf." [Glasses: "Wireless headset, about \$60."]

[Projector: a second laptop runs a buyer agent.] "That buyer found our store through a public catalog. Watch." [\$190 ... \$180 ... \$175 ... Deal.] "Nobody touched that negotiation, and the seller can't go below a floor the buyer never sees. That rule is code, not a prompt."

[Glasses speak: "Sold the PS4 for \$175. \$125 to go." Dashboard: SOLD \$175 · 1 item reused · 2 routed to donation/recycling · agent still working on 5.] "That last line is the product."

Then hand a judge the phone and let them press **Scout** themselves.

13. Setup checklist

Most of this can and should be done before the event. None of it is interesting, and all of it quietly eats four hours if left to hour zero.

Before the event — accounts

- ☐ **Apple Developer account** (paid program recommended) — one team for the whole project. Note the Team ID. Register the bundle ID now (e.g. `com.reloop.app`); it must match Xcode and the Wearables Developer Center and is painful to change.
- ☐ Add A's, B's and C's iPhones to the team (or add them as team members) so all three can install builds.
- ☐ Register at the Meta Wearables Developer Center; create the organization and a project for the iOS app; note **MetaAppID** and **ClientToken**. (Developer Mode can run without attestation, but wrong identifiers still fail to connect.)
- ☐ Set up a release channel and invite teammates' Meta accounts, in case you test outside Developer Mode.
- ☐ Dimensional: create an account at the Dimensional console / dimTELE and an API key (optional teleop fallback). Join their Discord.
- ☐ API keys: Anthropic (identification + agents), OpenAI (only if using DimOS's default agent model), Stripe test mode. Ask the Visa sponsor what sandbox/API access exists for the event.

Before the event — A, B, C (Mac + iPhone)

- ☐ Latest Xcode; Xcode command-line tools; iPhone in Developer Mode and deploying from Xcode.
- ☐ Clone `facebook/meta-wearables-dat-ios`; build and run the sample app on your iPhone.
- ☐ A: get **Mock Device Kit** running in the sample — the single highest-value pre-task for the glasses.
- ☐ A: add the Swift Package to the ReLoop project; fill in the MWDAT Info.plist keys, URL scheme, background mode and usage strings; confirm it builds.
- ☐ Wire the DAT MCP endpoint (<https://mcp.developer.meta.com/wearables>) and DAT coding guidance into your coding agent (the docs have Claude Code / Cursor / Codex pages).
- ☐ B + C: Python 3.12, uv, FastAPI, Anthropic Python SDK; the server repo builds; hello-world deployed to a real URL.
- ☐ B: price book researched for the 8 demo categories + 7 more, source and date per row. Identification prompt drafted against 8 demo photos and 3 dog-angle photos.
- ☐ C: skim the Anthropic tool-use docs and the Python MCP SDK README (for the MCP catalog module).

Before the event — D (no Mac; robot + web)

- ☐ Decide the DimOS machine: Ubuntu 22.04/24.04 (NVIDIA GPU recommended) via install, USB or dual-boot — or confirm Dimensional supplies one. macOS is experimental; WSL2 is not recommended.
- ☐ Install DimOS: `curl -fsSL https://raw.githubusercontent.com/dimensionalOS/dimos/main/scripts/install.sh | bash` (or `uv pip install 'dimos[base,unitree,sim]'` in a Python 3.12 venv).

- ☐ Run `dimos --replay run unitree-go2` (first run downloads ~75 MB) and `dimos --simulation run unitree-go2-agentic`; try `dimos mcp list-tools` and `dimos mcp call` against the sim.
- ☐ Write `ReLoopScout + scout.py` against the **simulation** so hour-5 work is a network swap, not new code.
- ☐ Read `DimOS AGENTS.md`, the Go2 setup doc, the Agents/skills doc and the CLI reference; point your coding agent at `AGENTS.md`.
- ☐ Web: dashboard + storefront skeleton renders from `fixtures/events.jsonl`.

Before the event — glasses & phone

- ☐ Meta AI app installed on the iPhone you'll demo with; glasses paired; glasses firmware and Meta AI app at or above the DAT **Version Dependencies** page minimums.
- ☐ Developer Mode on in the Meta AI app: Settings → App Info → tap the app version 5 times → toggle Developer Mode.
- ☐ Run the DAT sample app against the real glasses: registration, camera permission, one photo.

Demo props & power

- ☐ The 8 demo items; a low table or floor mat so several are at dog-camera height.
- ☐ Painter's tape for the dog zone; small spot labels (desk / shelf / floor corner / home).
- ☐ **Travel router** (dog + dog laptop + demo iPhone on a network you control).
- ☐ Dog charger + spare battery if available; glasses case charger; power strip; USB-C cable per laptop; phone hotspot as internet backup.

At the event — hour 0 (in parallel)

D — robot bring-up (~45–90 min, in the background):

- ☐ Ask Dimensional: model/firmware of the dog, which network it's on, whether we can provision it to our router, any venue rules for moving robots.
- ☐ Power on; `dimos go2tool discover`; `dimos go2tool connect-wifi --ssid ... --password ...`; `export ROBOT_IP=...`
- ☐ Pre-flight: `ping $ROBOT_IP < 10 ms`, 0% loss; onboard obstacle avoidance on.
- ☐ `dimos run unitree-go2` — camera visible in the command center (localhost:7779); a click-to-navigate works.
- ☐ Tape the zone. Name the spotter.

A — glasses bring-up (15 min, then back to the core):

- ☐ Glasses charged; sample app connects on the demo iPhone; one photo arrives. Then put them in the case until 7:00.

Everyone:

- ☐ Venue Wi-Fi works from every phone and laptop, including any captive portal; hotspot fallback tested.
- ☐ Phase 0 (section 6): schemas, fixtures, server deployed, `demo_loop` green, `iOS ContractsTests` green.

Standing hardware discipline

- Glasses back in the charging case between every test run. Dog on the charger whenever it isn't moving.
- One person holds the glasses at a time; one person commands the dog at a time. Everyone else tests against Mock Device Kit or DimOS simulation.
- Phones stay above 50%.

Reference links

- Meta DAT overview: <https://wearables.developer.meta.com/docs/develop/dat>
- DAT iOS integration:
<https://wearables.developer.meta.com/docs/develop/dat/build-integration-ios/>
- DAT setup / version dependencies / Mock Device Kit (iOS testing): linked from the DAT overview
- DAT iOS SDK + samples: <https://github.com/facebook/meta-wearables-dat-ios>
- DimOS: <https://github.com/dimensionalOS/dimos> · docs: <https://docs.dimensionalos.com>
- DimOS Go2 setup: [docs/platforms/quadruped/go2/setup.md](#) · Agents & skills: [docs/capabilities/agents/index.md](#) · CLI: [docs/usage/cli.md](#)